

Page Replacement Algorithms

FIFO, LRU and Optimal (OPT)

Prashant Jadav

DEPSTAR - CHARUSAT

Learning Objectives

- Explain why page replacement is required in demand-paged virtual memory.
- Simulate FIFO, LRU and Optimal (OPT) page replacement.
- Calculate page faults and page-fault rate for a reference string.
- Compare the algorithms in terms of implementation, performance and practicality.
- Solve GATE-style page-replacement numericals efficiently.

Why Page Replacement?

Situation

A process references a page that is not currently present in physical memory.

- ① A **page fault** occurs.
- ② If a free frame exists, load the required page into that frame.
- ③ If all frames are occupied, select a **victim page**.
- ④ Write the victim back to disk if it is dirty.
- ⑤ Load the required page and restart the interrupted instruction.

Key question

Which page should be removed when no free frame is available?

FIFO, LRU and OPT answer this question using different rules.

Common Terminology

Reference string: Sequence of page numbers requested by the CPU.

Frame: A fixed-size physical-memory slot that can hold one page.

Page hit: Requested page is already in a frame.

Page fault: Requested page is absent from all frames.

Victim: Page selected for replacement after a fault when memory is full.

Page-fault rate:

$$\text{Page-fault rate} = \frac{\text{Number of page faults}}{\text{Total number of page references}}$$

FIFO Page Replacement

Rule

Replace the page that has been in memory for the **longest time**.

- Maintains pages in arrival order.
- Conceptually implemented using a queue.
- On replacement: remove the page at the front and insert the new page at the rear.
- Does not consider how recently or frequently a page was used.

Strength

Very simple and low implementation overhead.

Limitation

FIFO can suffer from **Belady's anomaly**: increasing the number of frames can sometimes increase the number of page faults.

LRU Page Replacement

Rule

Replace the page that has **not been used for the longest time in the past**.

- Uses recent-reference history.
- Exploits the principle of **temporal locality**.
- On every reference, the recency information must be updated.
- Practical implementations may use timestamps, counters, stack/list structures, or hardware-assisted mechanisms.

Important

LRU is a **stack algorithm**; increasing the number of frames does not increase its page-fault count for the same reference string.

Optimal (OPT / MIN) Page Replacement

Rule

Replace the page whose **next reference is farthest in the future**.

- If a page will never be referenced again, it is an ideal victim.
- OPT produces the minimum possible number of page faults for a given reference string and number of frames.
- It requires knowledge of future references.
- Therefore, it is generally **not implementable** as a real-time replacement policy.

Use in practice

OPT is mainly a theoretical benchmark used to evaluate other page-replacement algorithms.

FIFO vs LRU vs OPT

Feature	FIFO	LRU	OPT
Victim rule	Oldest arrival	Least recently used	Farthest future use
Uses past history	No	Yes	No
Needs future knowledge	No	No	Yes
Implementation	Simple	More complex	Not practical
Belady anomaly	Possible	No	No
Typical role	Simple baseline	Practical policy family	Theoretical lower bound

OPT faults $\leq$ LRU/FIFO faults for a fixed reference string and frame count

The exact relationship between FIFO and LRU depends on the reference string.

Worked Numerical Example

Reference string:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2

Number of frames: 3, initially empty.

Task

Calculate the number of page faults using FIFO, LRU and OPT.

- Mark **F** when the requested page is absent.
- Mark **H** when the requested page is already present.
- Replacement starts only after all three frames are occupied.

Worked Example – FIFO

Reference	7	0	1	2	0	3	0	4	2	3	0	3	2
Frame 1	7	7	7	2	2	2	2	2	4	4	4	0	0
Frame 2	-	0	0	0	0	3	3	3	2	2	2	2	2
Frame 3	-	-	1	1	1	1	0	0	0	3	3	3	3
Fault?	F	F	F	F	H	F	F	F	F	F	F	H	H

FIFO replacement sequence:

$7 \rightarrow 0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 0$

Result

FIFO page faults = 10

Worked Example – LRU

Reference	7	0	1	2	0	3	0	4	2	3	0	3	2
Frame 1	7	7	7	2	2	2	2	4	4	4	0	0	0
Frame 2	-	0	0	0	0	0	0	0	2	2	2	2	2
Frame 3	-	-	1	1	1	3	3	3	3	3	3	3	3
Fault?	F	F	F	F	H	F	H	F	F	F	F	H	H

Replacement decisions

After page 3 arrives, page 1 is the least recently used.

After page 4 arrives, page 3 is the least recently used.

Later, page 2 and page 3 cause further LRU replacements based on most-recent reference times.

Result

LRU page faults = 9

Worked Example – OPT

Reference	7	0	1	2	0	3	0	4	2	3	0	3	2
Frame 1	7	7	7	2	2	2	2	2	2	2	2	2	2
Frame 2	-	0	0	0	0	0	0	0	4	4	0	0	0
Frame 3	-	-	1	1	1	3	3	3	3	3	3	3	3
Fault?	F	F	F	F	H	F	H	F	H	H	F	H	H

OPT decisions

When page 2 arrives, page 7 is never referenced again, so evict 7.

When page 4 arrives, page 1 is never referenced again, so evict 1.

Result

OPT page faults = 7

Worked Example – Final Comparison

Algorithm	Page Faults	Page-Fault Rate
FIFO	10	$\frac{10}{13} \approx 76.9\%$
LRU	9	$\frac{9}{13} \approx 69.2\%$
OPT	7	$\frac{7}{13} \approx 53.8\%$

Exam observation

For the same reference string and frame count, OPT provides the benchmark minimum. FIFO and LRU must be simulated according to their own rules; do not assume one always beats the other.

How to Solve Page-Replacement Numericals

- 1 Write the reference string horizontally.
- 2 Draw one row for each available frame.
- 3 Start with all frames empty.
- 4 For every reference:
 - If page is present: **Hit**; no replacement.
 - If page is absent and a frame is free: load it; **Fault**.
 - If page is absent and memory is full: select the victim using FIFO/LRU/OPT.
- 5 Count all faults, including faults that fill initially empty frames.

Common mistake

Do not count a replacement as an additional fault. A replacement occurs *because* the current reference has already caused one page fault.

Question: Which page-replacement algorithm can have a higher page-fault rate when the number of allocated frames is increased?

- ① LRU
- ② OPT
- ③ MRU
- ④ FIFO

Answer

FIFO

Concept tested: *Belady's anomaly*. FIFO does not satisfy the stack/inclusion property, so increasing the number of frames can sometimes increase the number of faults.

Source: GATE CSE 2016 Set 2, Question 20.

Reference string:

1, 2, 3, 2, 4, 1, 3, 2, 4, 1

Frames: 3, initially empty.

The question asks for the relationship among the number of page faults under LRU, FIFO and OPT.

Solution idea

Simulate all three algorithms using the same reference string and three frames.

The resulting relationship is:

$$\text{OPT} < \text{FIFO} < \text{LRU}$$

Concept tested: Direct comparison of page-replacement policies through page-fault simulation.

Source: GATE CSE 2009, Operating Systems / Memory Management.

GATE PYQ – 2014 CSE Set 2

A system has 20 physical page frames initially containing pages 101–120. A program accesses pages 1–100 in order and repeats this sequence three times.

Question

Which replacement policy experiences the same number of page faults as OPT?

Options include:

- LRU
- FIFO
- LIFO
- MRU

Answer

MRU

For this cyclic sequential-reference pattern, MRU has the same fault count as OPT.

Source: GATE CSE 2014 Set 2, Question 33.

Belady's Anomaly

Definition

Belady's anomaly is the situation in which increasing the number of available page frames causes the number of page faults to **increase**.

- Classical example: FIFO.
- LRU and OPT are stack algorithms and do not exhibit this anomaly.
- GATE frequently tests this as a conceptual MCQ.

More frames $\nrightarrow$ fewer FIFO faults in every case

Exam cue

If a question asks which common page-replacement algorithm can show Belady's anomaly, think **FIFO**.

Practice Numerical 1

A process has **3 page frames**, initially empty.

Reference string:

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

- 1 Calculate FIFO page faults.
- 2 Calculate LRU page faults.
- 3 Calculate OPT page faults.
- 4 Determine the page-fault rate for each algorithm.

Answer check

FIFO = 9, LRU = 10, OPT = 7

Practice Numerical 2 – GATE Style

A demand-paged system has **4 page frames**, initially empty.

Reference string:

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

- ① Count the page faults using FIFO.
- ② Count the page faults using LRU.
- ③ Count the page faults using OPT.
- ④ Explain why the three counts differ.

Tip

For OPT, inspect only the future references of the pages currently in memory and evict the one whose next use is farthest away.

Key Takeaways

- **FIFO:** remove the oldest page; simple but can exhibit Belady's anomaly.
- **LRU:** remove the least recently used page; exploits temporal locality.
- **OPT:** remove the page whose next use is farthest in the future; gives the minimum possible fault count.
- Always count initial page loads as page faults.
- For numericals, construct the frame table carefully and process the reference string one reference at a time.

OPT is the benchmark; LRU is a practical approximation; FIFO is the simplest baseline.